

Context & Harness Engineering

Lessons from AKD

Nish (Nishan Pantha) · NASA-IMPACT

2nd ESA-NASA International Workshop on AI Foundation Models for Earth Observation

Day 4 / Track 1 — GeoAI Agent Tutorial · May 2026

The bottleneck has moved

"Context engineering is the delicate art and science of filling the context window with just the right information for the next step."

— *Andrej Karpathy, June 2025*

The **model** isn't the bottleneck. What's around it is — the **context** that goes in, and the **harness** that wraps it.

CONTEXT ENGINEERING

HARNESS ENGINEERING

Prompt engineering → context engineering

Prompt engineering → context engineering

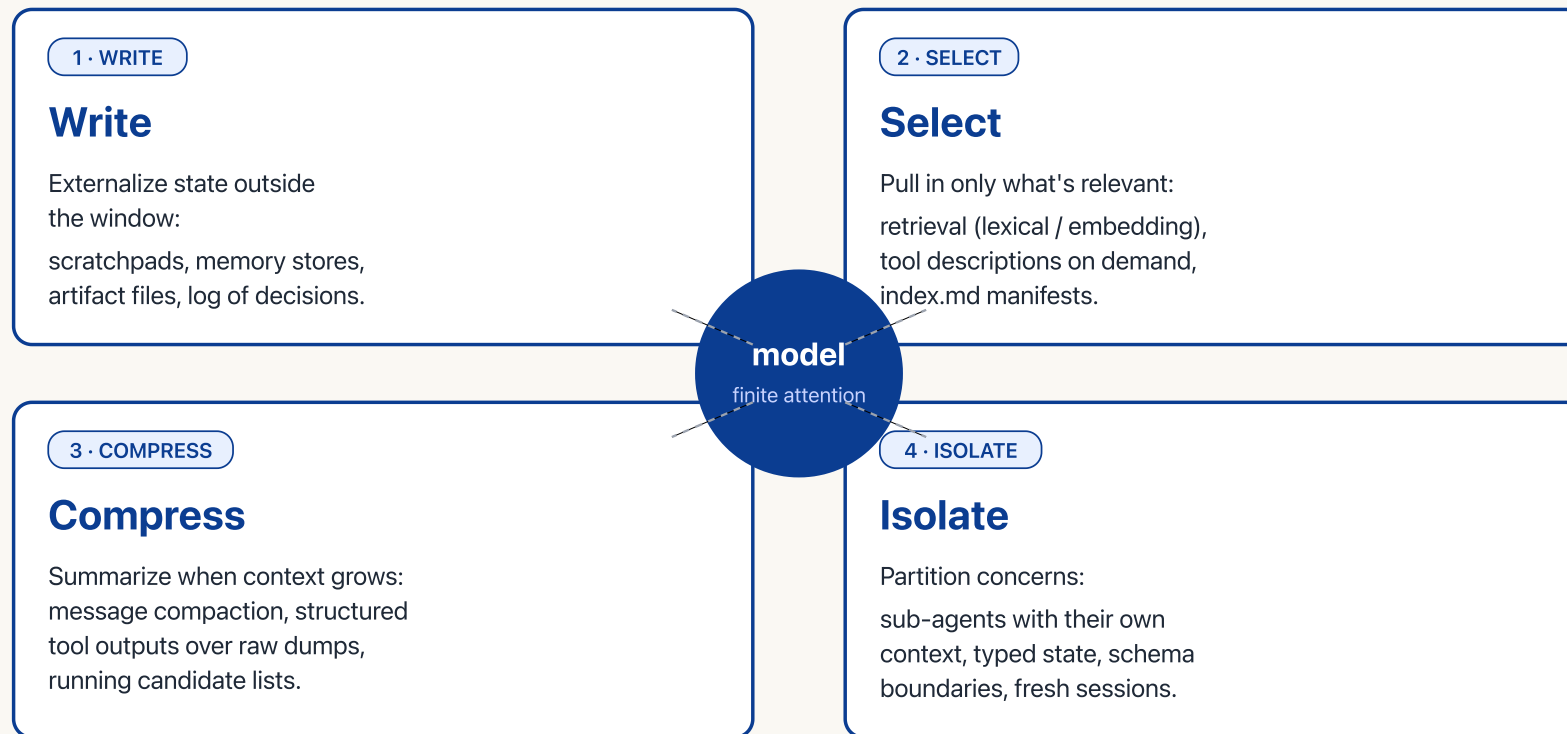
a string becomes a system — what changes when you scale to scientific agents

PROMPT ENGINEERING	CONTEXT ENGINEERING
a clever string	a system that curates
artefact one prompt template; you tweak words.	artefact a directory of files + a tool surface + guardrails.
unit of work a single LLM call.	unit of work a run: write → select → compress → isolate, then call.
scaling "add another paragraph."	scaling retrieve / compact / partition across sub-agents.
failure mode wrong wording → wrong answer.	failure mode poisoning · distraction · confusion · clash · context rot.
evaluation vibes / spot-check.	evaluation benchmark suites · trajectory evals · reproducibility logs.

The four pillars

Context engineering — four pillars

filling the context window with the right information, every step



attributed to LangChain — the canonical decomposition

How contexts fail

Bigger window ≠ better answers

four ways contexts fail in production

01 Poisoning

A hallucination, bad tool output, or stale fact lands in context — then propagates downstream as if it were ground truth.

02 Distraction

Window so full of "relevant" history that the model loses sight of what the user actually asked. Attention budget gone.

03 Confusion

Irrelevant tool outputs, dead references, or stray instructions muddle the next decision. Signal drowned by noise.

04 Clash

Two pieces of context disagree. System prompt says one thing, retrieved doc the other. The model picks — sometimes wrong.

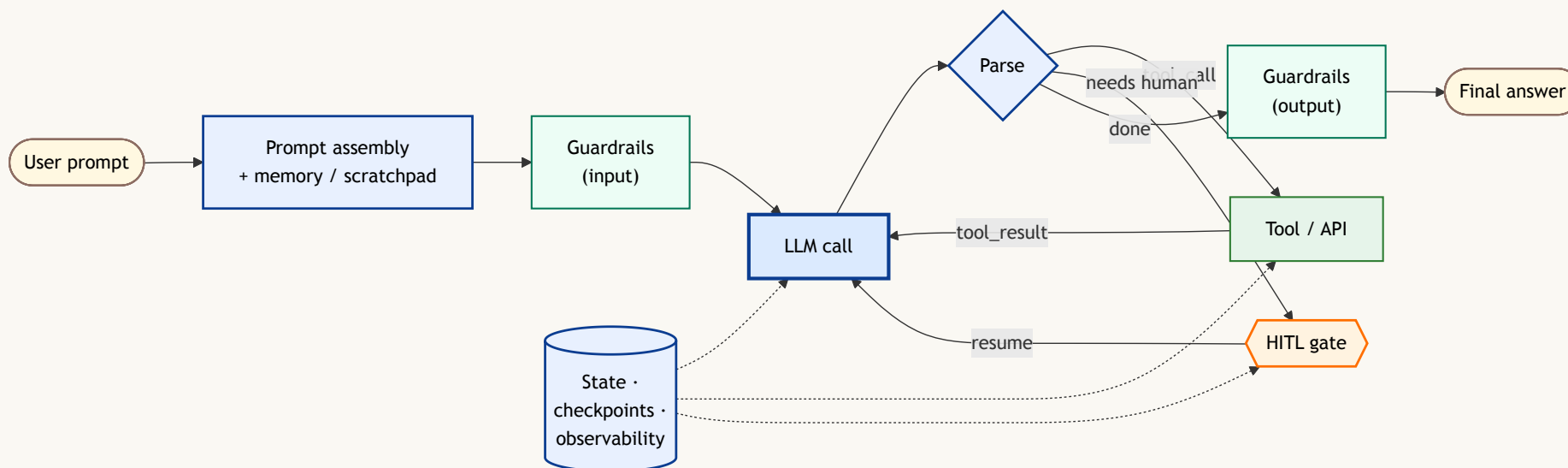
Context rot

Chroma (2025): every frontier model degrades as input grows. Long ≠ thoughtful.
Treat the window as a finite attention budget, not free storage.

refs: Modarressi et al. "NoLiMa" arXiv:2502.05167 (2025) · Chroma "Context Rot" (2025) · dbreunig.com "How Long Contexts Fail" (2025)

Harness engineering

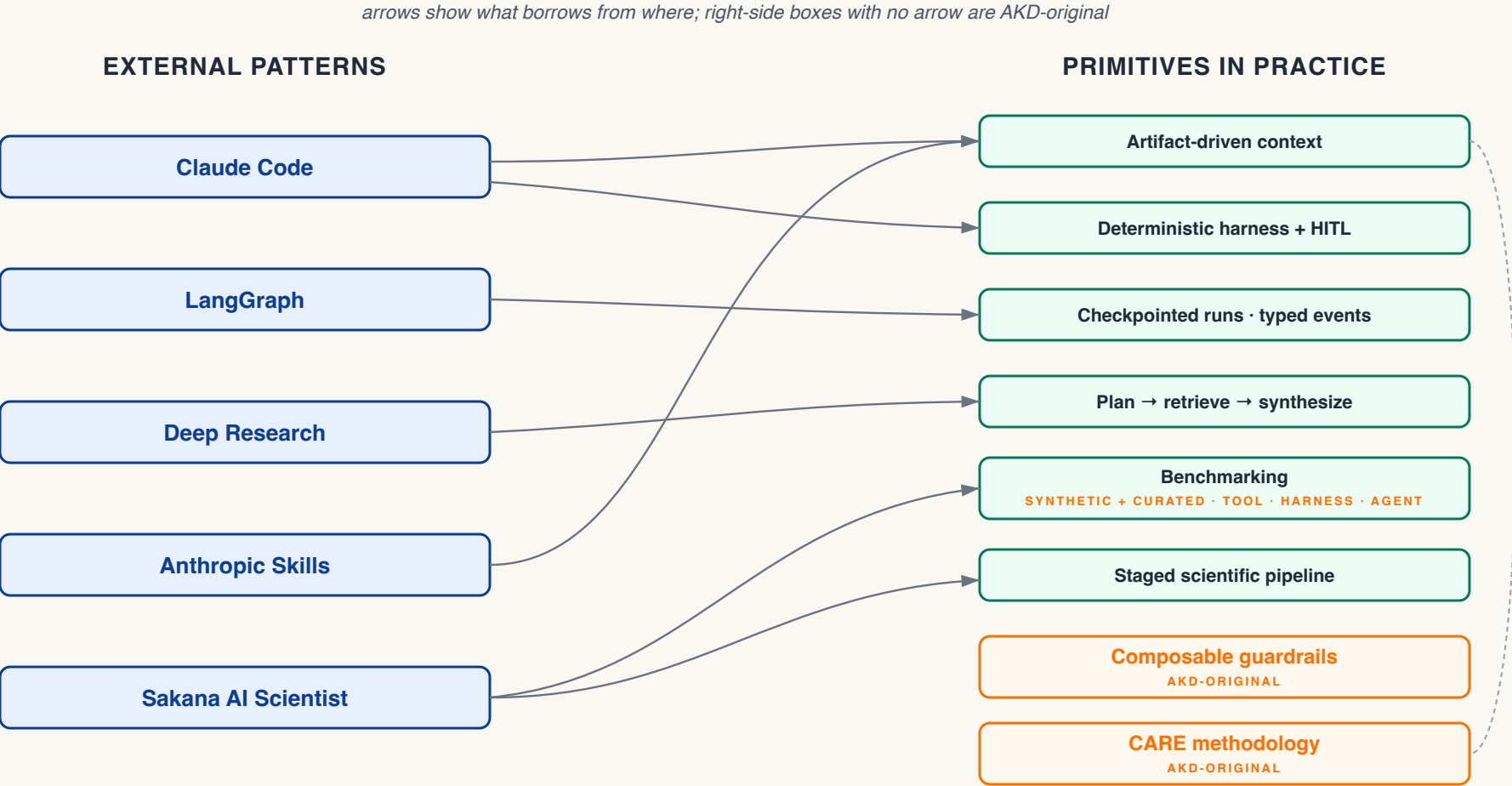
"~98% of Claude Code is deterministic infrastructure." — the agent loop is a while-loop; the harness does the work.



A capable model + a poor harness loses to a weaker model + a great harness. Reliability lives in the surrounding **deterministic** code — tools, loop, guardrails, state.

"Agents are only as useful as what they can connect to." — Anthropic, 2025

Community patterns



composable guardrails — input + output (safety · risk · factuality · attribution); chain with and · or · fail-fast operators.

Artifacts as the knowledge layer

Anthropic Skills — the spec

a SKILL.md manifest + supporting subdirectories, packaged as a directory

my-skill/

SKILL.md

MANIFEST

```
---
name: my-skill
description: what this skill does
---
# body: instructions for the agent
```

references/

retrievable knowledge — loaded on demand
api-reference.md · domain-glossary.md · policy.md

scripts/

executable hooks — the agent runs them as tools
validate.py · fetch.sh · format.ts

assets/

supporting files — examples, schemas, fixtures
examples.json · schema.yaml · fixtures/

WHY THE SPEC WORKS

load on demand

SKILL.md is the manifest the agent reads;
other files load only when relevant — no
forced context bloat.

files, not code

behavior is in markdown + scripts —
diff-able, reviewable, versioned, swappable
across models.

tools follow the spec

when the spec is consistent and well-typed,
tool calls become predictable — the agent
stops guessing.

portable across runtimes

Claude Code packages this same way;
CLAUDE.md + supporting files is the
same pattern, earlier.

Get the spec right and behavior follows with consistency.

Harness primitives in practice

The harness stack

primitives that wrap every published agent — built once, reused everywhere

LLM

GPT-5 / Claude / Granite — chosen per agent, swappable in config

streaming-native

every step emits a typed event;
the UI never polls.

BaseAgent · BaseTool

schema-first: Pydantic input / output / config schemas
`class DomainAgent(BaseAgent[InputSchema, OutputSchema]): ...`

deterministic harness

~the model loop is a while-loop;
the value lives in the surrounding
deterministic code.

Typed StreamEvents

starting · running · thinking · tool_calling · tool_result ·
partial_output · human_input_required · completed · failed

composable guardrails

attach as *input* or *output* of any agent;
three providers swappable per agent.

HITL as a tool

HumanTool raises → run pauses → state checkpointed → resume with human's answer

HITL as first-class

approval gates are events, not
UI hacks. Survive process restart.

Composable guardrails (operators: and · or · fail-fast)

`input ▷ safety_guardrail (HAP · policy · injection)`
`output ▷ risk_guard >> factuality (chained, swappable)`
Attribution · Factuality · Groundedness

checkpointed runs

LangGraph + Postgres persists
workflow state for hours / days.

LangGraph · PostgreSQL checkpoints · FastAPI · SSE

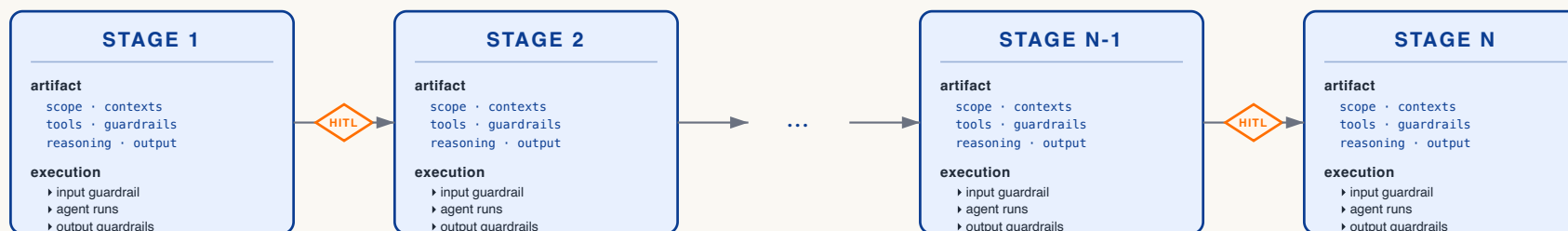
Composable guardrails — attach as *input* (HAP, policy, prompt-injection) or *output* (risk, factuality, attribution); chain with **and** · **or** · **fail-fast** operators; providers swappable per agent.

Multi-agent stance — single thread per agent; multi-agent only at workflow boundaries, with explicit hand-offs and HITL gates.

End-to-end: staged workflows in production

Two workflows, one staged pattern

each stage carries its own artifact · context · guardrails; explicit hand-offs; HITL gates where needed



INSTANCES LIVE TODAY

- **CM1** — weather computational modeling
- **FM Inference** — EO foundation-model inference

same harness pattern, two scientific workflows — deployed today

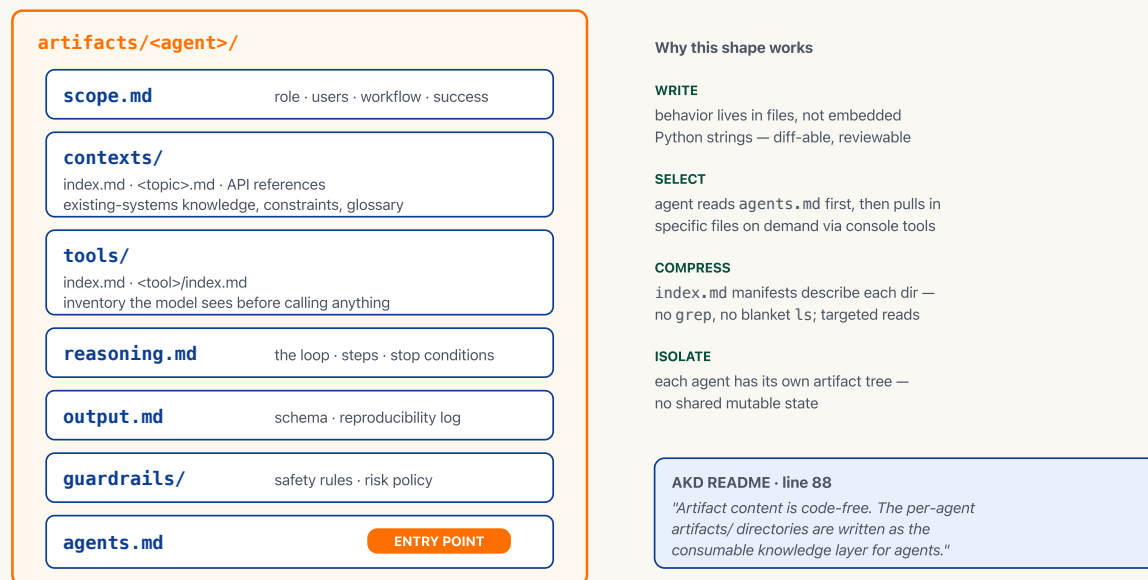
Both AKD workflows follow the same **staged pattern** — each stage carries its own *artifact*, *context*, *guardrails*; hand-offs are explicit; approval gates where needed.

Instances live today — CM1 (weather computational modeling) · FM Inference (EO foundation-model inference)

AKD today — a case study

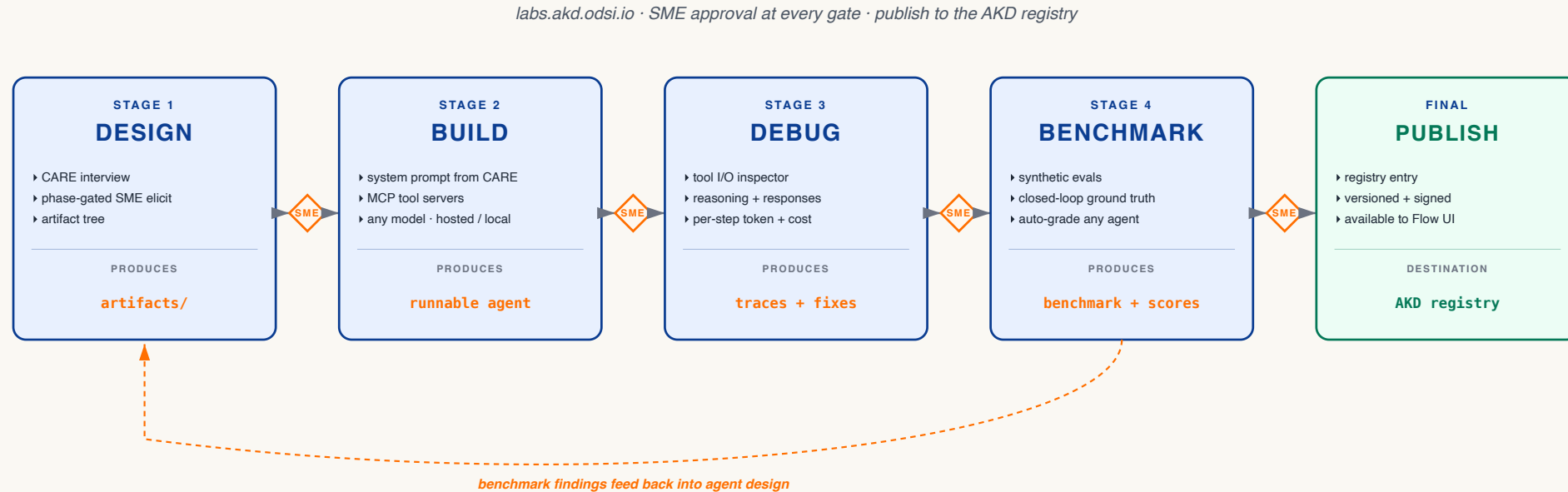
Artifact-driven context — AKD & the tutorial agent share this shape

behavior is written, not coded · each file is consumable knowledge for the LLM



CARE produces this artifact tree. Same pattern as Anthropic Skills — `scope.md`, `contexts/`, `tools/`, `guardrails/`, `reasoning.md`, `output.md`, with `agents.md` as the entry-point manifest. 6 domain agents · 2 closed-loop workflows (CM1, FM Inference) · live across Earth Science · Astrophysics · Planetary Science · flow.akd.odsio

AKD Labs — the 4-stage workbench



Every published agent passes through the same 4-stage workbench — **SME approval at every gate, publish to the AKD registry**. Context · harness · guardrails · evaluation, all wired in.

Takeaways

Three things to take away

1. **Own your tools and harness.**
2. **Artifact-driven context** — write the agent's knowledge as files, not code.
3. **Evaluation closes the loop** — without benchmarking, "context engineering" is just vibes.

Tie-back to today's tutorial

The Prithvi agent you'll run loads from an **artifact** with `scope.md / contexts/ / tools/ / guardrails/ / reasoning.md / output.md / agents.md`.

Same shape as the production patterns we just covered — applied to EO foundation models.

Read: Anthropic *Effective context engineering* · Cognition *Don't build multi-agents* · dbreunig.com *How Long Contexts Fail* ·

Chroma *Context Rot* · OpenAI *Harness engineering*

Thank you · questions? — `github.com/NASA-IMPACT/akd-suite` · `labs.akd.odsi.io`